Assignment 2: Software Quality III (CSCN73010), Build and Release

NAME **Amna Farid**

ROLL NUMBER **9040297**

1. GitHub repository

https://github.com/crispyT-T/Flask_Image_Recognition

I forked Sachin-crypto/Flask_Image_Recognition and based my branch on commit `62d8114` ("Added dl model", 29 April 2023). That is the last commit which holds only the legacy application. The commits after it already contain a finished version of this assignment written by someone else, so I left them out.

---

2. Steps to set up and run the application

The project needs Python 3.11.

```
git clone https://github.com/crispyT-T/Flask_Image_Recognition.git
cd Flask_Image_Recognition
python3.11 -m venv .venv
source .venv/bin/activate
pip install -r requirements.txt
python app.py
```

The application then serves on http://127.0.0.1:9000. Pick an image from `test_images/`, submit it, and the predicted digit appears.

Installation problems I ran into

The pinned dependencies would not install. The original `requirements.txt` asked for `tensorflow==2.10.0`. On Apple Silicon that fails straight away:

```
ERROR: Could not find a version that satisfies the requirement tensorflow==2.10.0
       (from versions: 2.13.0rc0, ... 2.22.0rc0)
ERROR: No matching distribution found for tensorflow==2.10.0
```

TensorFlow 2.10 came out before arm64 macOS wheels were published under the `tensorflow` name, so there is simply no build for this platform. The oldest one available is 2.13.

I moved to TensorFlow 2.15.1. That version matters for a specific reason: it is the last release that still bundles Keras 2. The saved model is a Keras 2 HDF5 file, and Keras 3, which ships with TensorFlow 2.16 onwards, will not load that format unless you add the `tf-keras` compatibility package. I also held NumPy at 1.26.4, since TensorFlow 2.15 is not built against NumPy 2, and Werkzeug below 3.0, because Flask 2.2 imports things that Werkzeug 3 removed.

The model path pointed at the original author's machine. `model.py` contained this line:

```
model = load_model("D:/SACHIN/Models/Hand-Sign-Digit-Language/digit_model.h5")
```

Importing the module on any other computer raised `OSError`. The model file `digit_model.h5` is committed at the root of the repository, so I changed the path to be worked out relative to the module itself:

```
MODEL_PATH = os.path.join(os.path.dirname(__file__), "digit_model.h5")
model = load_model(MODEL_PATH)
```

Editor and cache files were being tracked. The repository had `.idea/` and `__pycache__/` committed. I removed both and added a `.gitignore`.

---

3. Screenshots of the application running

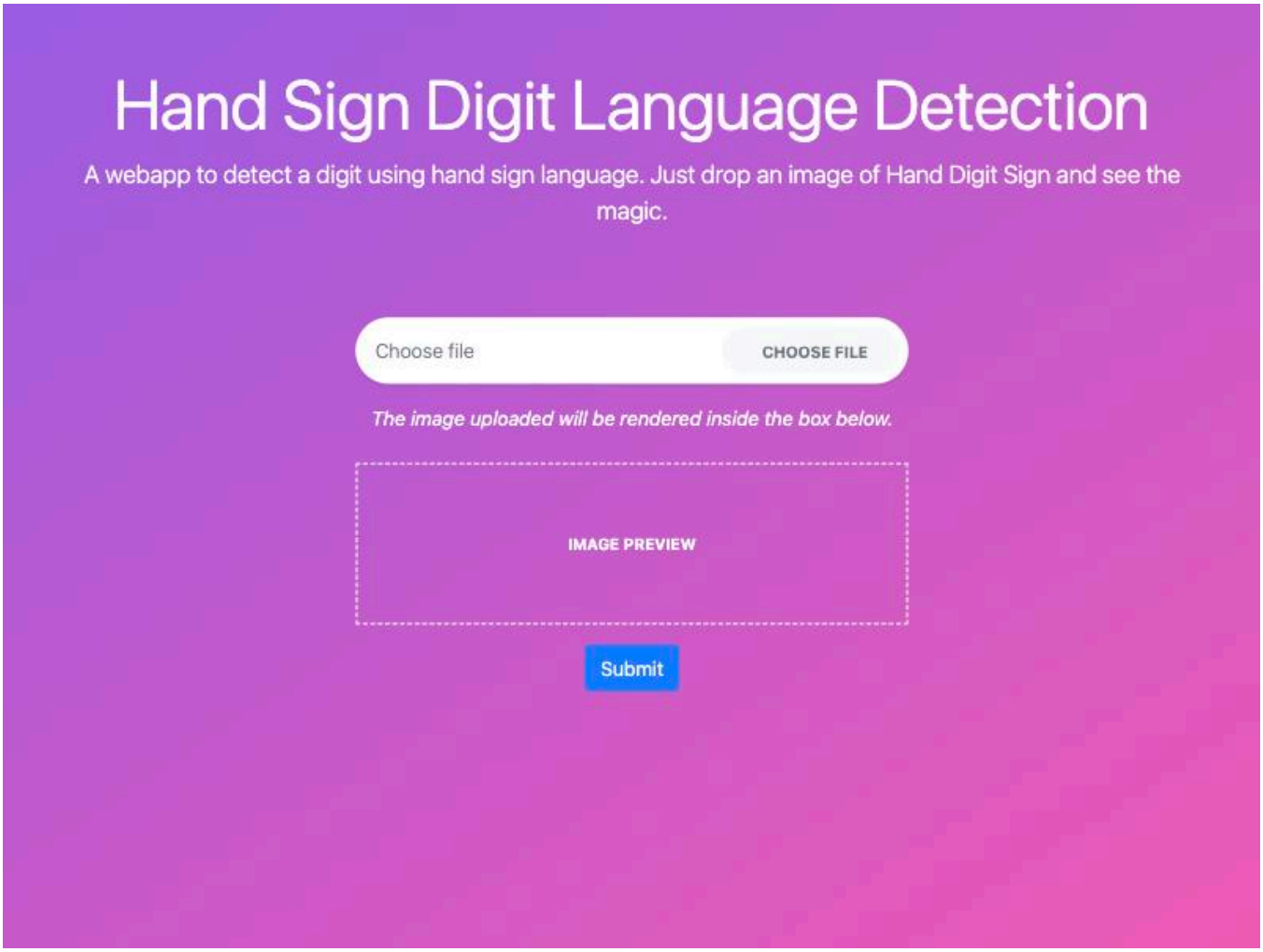


Figure 1. The home page at `http://127.0.0.1:9000/`.

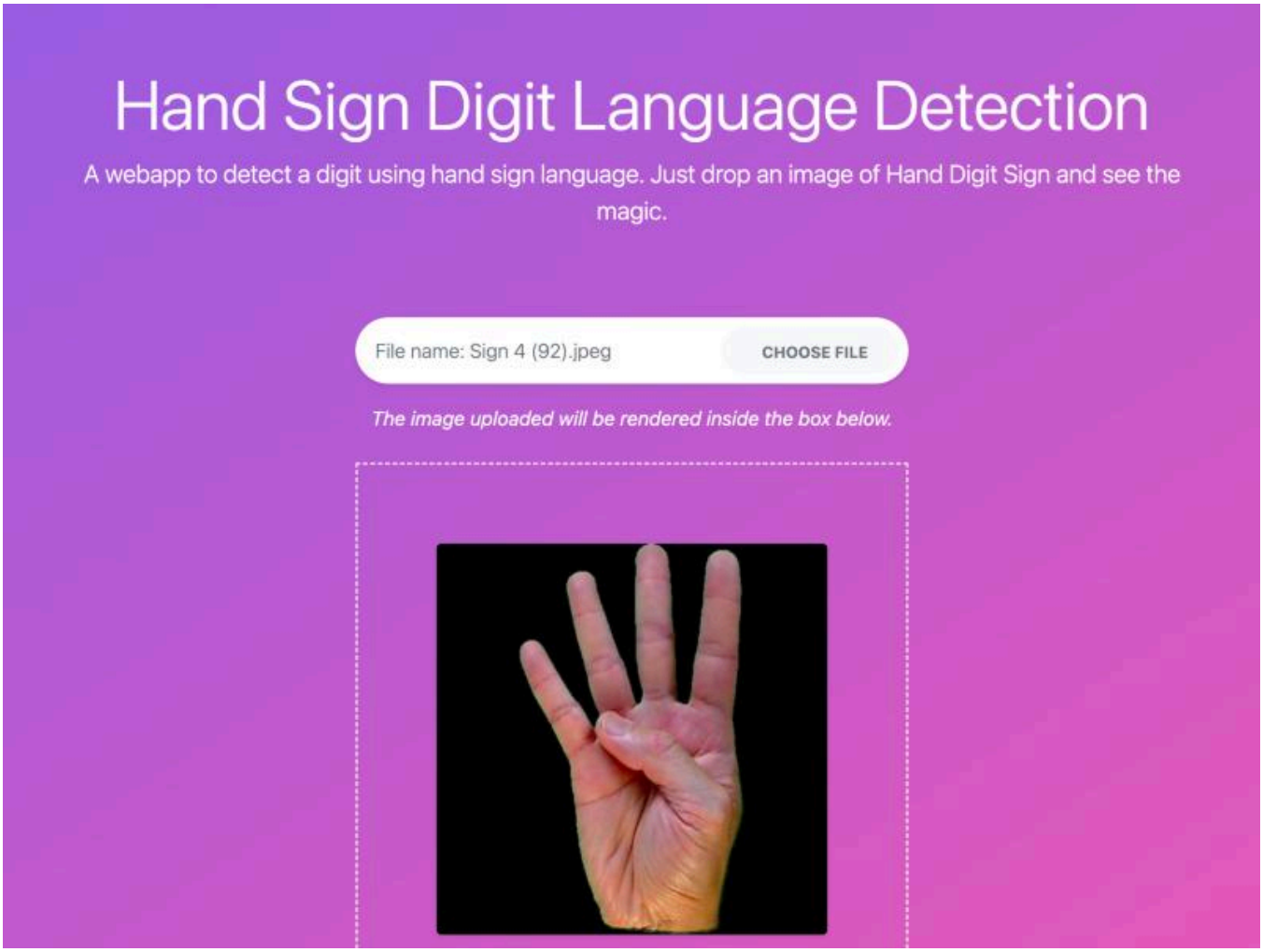


Figure 2. An image chosen. The preview and the file name are produced by `static/js/image_upload.js` in the browser. Nothing has been sent to the server yet.



Figure 3. The result page. A hand sign for 4 comes back as `4`.

4. Main application components

FILE	PURPOSE
`app.py`	The Flask entry point. It creates the application and defines both routes. `/` serves the upload form and `/prediction` takes the file and returns the digit.
`model.py`	The machine learning layer. It loads the Keras model when the module is imported, and holds `preprocess_img`, which turns an upload into an array the model can read, and `predict_result`, which runs the prediction.
`digit_model.h5`	The trained model. 28 MB, Keras 2 HDF5 format, ten output classes.
`templates/layout.html`	The Jinja2 base template. It loads Bootstrap 4.3 and jQuery from a CDN, plus the project's own CSS and JavaScript.
`templates/index.html`	The upload page. A `multipart/form-data` form with a file input named `file`.
`templates/result.html`	Shows `predictions` when the prediction worked and `err` when it did not.
`static/css/custom.css`	The gradient background and the styling on the upload area.
`static/js/image_upload.js`	Runs only in the browser. It previews the chosen image and shows its file name.
`test_images/`	100 sample images, ten for each digit.

Flask is configured in `app.py` with `app = Flask(__name__)`. There is no config object or settings file, so every default applies, which is how Flask finds `templates/` and `static/`. The port (9000) and the debug flag are set inside the `__main__` guard, so they only take effect when the file is run directly.

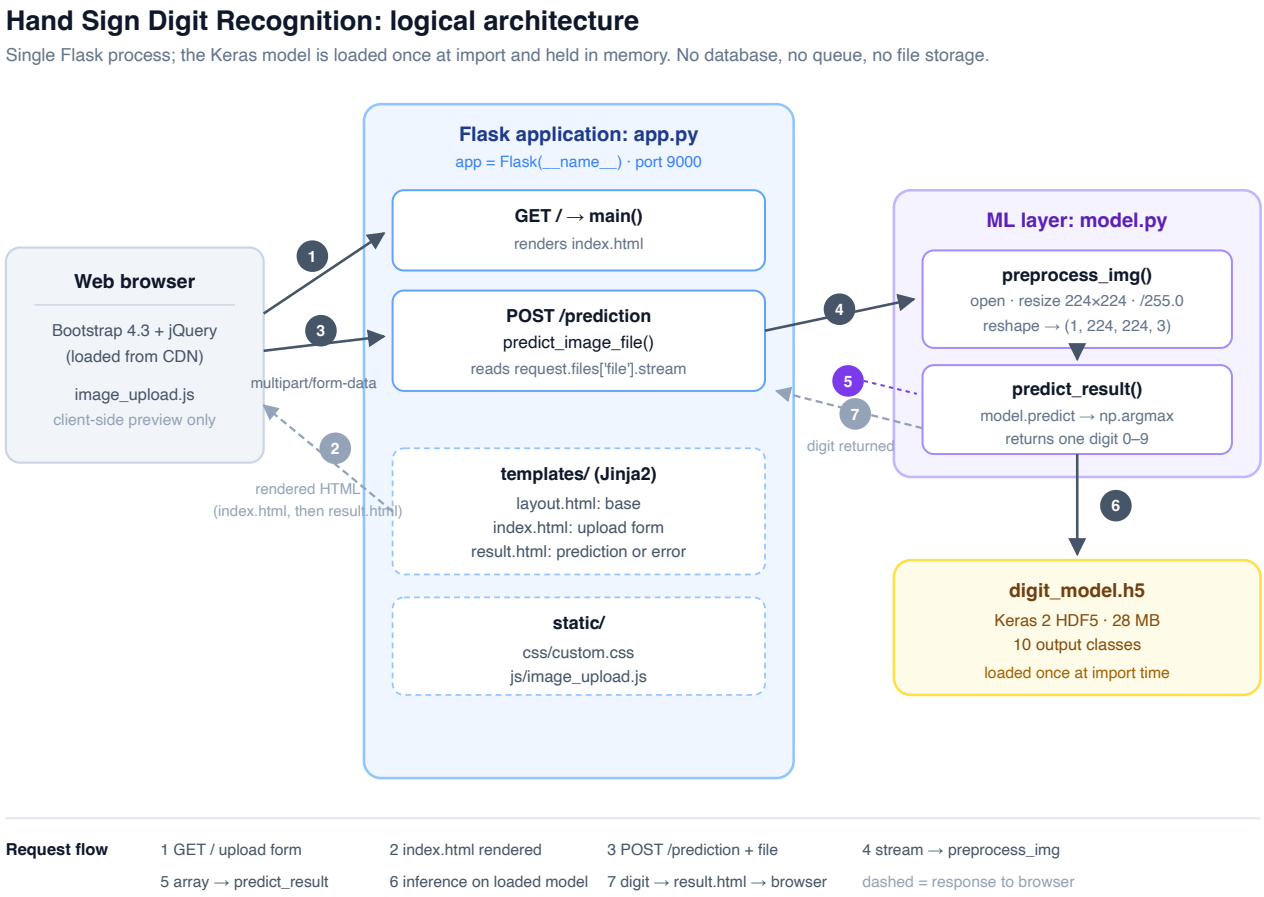
The model is loaded in `model.py` at module level rather than inside a function. This turned out to be the most important detail in the whole codebase. Because the call sits at the top of the file, the full 28 MB model is read and the network built as soon as anything imports `model`, which includes `app.py` at startup and any test file. It is then kept in a module level variable that `predict_result` uses. The advantage is that a request does not have to wait for the model to load. The disadvantage is that the module cannot be imported at all unless the model file is present, and that is what made mocking necessary in the test suite.

Images are preprocessed by `preprocess_img`. It opens the file with Pillow, resizes it to 224x224, scales the pixel values from 0-255 down to 0-1, and reshapes the result into a batch of one. The parameter is called `img_path`, but `app.py` actually passes a file stream rather than a path. Pillow accepts either, so this is confusing rather than broken.

Templates and static resources sit where Flask expects them, in `templates/` and `static/`, both relative to `app.py`.

An upload turns into a prediction like this. The browser asks for `GET /` and `main()` renders `index.html`. The user picks a file, which `image_upload.js` previews locally. Submitting the form posts `multipart/form-data` to `POST /prediction`. There, `predict_image_file()` hands `request.files['file'].stream` to `preprocess_img`, which returns a `(1, 224, 224, 3)` array. `predict_result` calls `model.predict` on it, and `np.argmax` reduces the ten class probabilities down to a single digit. That digit goes into `result.html`. If any step raises an exception, the error page is shown instead.

5. Architecture diagram



The design is as simple as it looks. One Flask process holds the model in memory and serves a browser over HTTP. There is no database, no queue, no separate model server and nothing stored on disk. The uploaded file is read from the request stream and then discarded.

6. Pylint configuration and code quality improvements

Configuration

I kept `.pylintrc` short on purpose. My aim was to leave Pylint's defaults alone and fix whatever it reported, rather than switch checks off to push the score up.

SETTING	VALUE	REASONING
`ignored-modules`	`keras`, `tensorflow`	Both libraries build a lot of their public API lazily when imported. Static analysis cannot see those attributes, so Pylint reports `no-member` errors that are not real.
`fail-under`	`9.0`	Pylint exits with a non-zero code below this, which means a drop in quality fails the build instead of just printing a warning.
`max-line-length`	`100`	This is Pylint 3's own default. I wrote it out so the value is visible rather than inherited silently.

The only message I disabled outright is `duplicate-code`. One other message is suppressed on a single line in `conftest.py`, with the reason next to it: `from app import app # pylint: disable=import-outside-toplevel`. Importing `app` at the top of that file would load the model before the patch that replaces the loader is in place, which would defeat the point of the fixture.

What I changed

The first run scored **7.50/10**.

FINDING	WHAT I DID
`C0114`, `C0116`, six missing docstrings	Wrote them. The one on `model.py` records that the model loads once per process. The one on `preprocess_img` explains that it takes a path or a stream, since the parameter name implies only a path.
`W0702`, a bare `except:`	Replaced it with `except (KeyError, OSError, ValueError)`, one exception for each failure a bad upload actually causes, and added `app.logger.exception` so the real error reaches the log. The bare version also caught `KeyboardInterrupt` and `SystemExit`, which meant the server could not be stopped cleanly while handling a request.
`R1710`, inconsistent return statements	The body was wrapped in `if request.method == 'POST'`, which the route decorator already guarantees. If that check had ever been false the view would have returned `None`, which Flask rejects. Removing the check removed the problem.

The final score is **10.00/10**.

The score itself turned out to be the least useful thing Pylint told me. Adding six docstrings moved it from 7.50 to 9.38 without changing how the program behaves at all. The two findings that genuinely mattered moved it the least. The bare `except` was hiding the cause of every failure, and the inconsistent return left a real path where the view returned nothing. If I had treated the number as the goal, I would have done the easy work first and stopped there.

---

7. Pre-commit configuration

These hooks run before each commit once `pre-commit install` has been run:

HOOK	PURPOSE
`trailing-whitespace`, `end-of-file-fixer`	Basic whitespace tidying. The first run fixed missing newlines at the end of four templates and the stylesheet.
`check-yaml`	Checks the Actions workflow is valid YAML before it reaches CI.
`check-added-large-files` (`--maxkb=30720`)	The limit sits just above `digit_model.h5` at 28 MB, so the model can be committed but nothing bigger gets in by mistake.
`check-merge-conflict`	Stops conflict markers being committed.
`debug-statements`	Stops a stray `breakpoint()` or `pdb` import being committed.
`pylint` (local hook)	Runs Pylint using the active interpreter, so it resolves Flask and Keras the same way CI does.

---

8. Pytest test cases

There are nineteen tests, split between `tests/test_routes.py` and `tests/test_model.py`.

REQUIREMENT	TESTS
Home page accessible	`test_home_page_is_served`
Valid request gives the expected response	`test_valid_upload_returns_the_predicted_digit`, which also checks the model was given exactly one correctly shaped and normalised batch
Missing uploaded file	`test_missing_file_is_rejected`
Invalid or unexpected input	`test_non_image_upload_is_rejected`, `test_empty_filename_is_rejected`, `test_greyscale_image_is_rejected`, `test_get_is_not_allowed_on_the_prediction_route`
Preprocessing and prediction	Nine tests covering shape, dtype, normalisation, path and stream inputs, and the argmax step

Edge cases

I included a 1x1 image and a 4000x10 image, so very odd aspect ratios still have to resize to 224x224, and an image that is already 224x224, where the resize does nothing and must not break anything. A greyscale image is a useful case because it preprocesses to one channel, so the reshape to three channels raises `ValueError` and the application rejects it instead of predicting from the wrong data. An empty filename covers what a browser sends when the form is submitted with no file chosen. One test points the stub at class 9 rather than 4, which proves the answer follows the model instead of being hardcoded. A last test checks the stub itself still looks like real model output, so the suite cannot quietly end up testing something meaningless.

Mocking

Since `model.py` loads the model when it is imported, every test run would otherwise spend time reading 28 MB before collecting a single test. To avoid that, `conftest.py` patches `keras.models.load_model` as soon as it is imported, which happens before any test module imports `app` or `model`. The application therefore gets a stub whose `predict` returns a fixed set of ten probabilities. The whole suite runs in **0.44 seconds**.

One test is marked `slow` and uses the real model. It keeps a reference to the original loader from before the patch, loads `digit_model.h5` through it, and checks that a genuine sample of the digit 4 is classified as 4 with more than 50% confidence. It also checks the object it got back really is a Keras model. That check matters, because the stub also returns a vector peaking at class 4, so without it the test would pass whether or not the real model had been loaded.

---

9. Evidence of local Pylint and pytest execution

```
$ pylint app.py model.py conftest.py tests/


--------------------------------------------------------------------
Your code has been rated at 10.00/10 (previous run: 9.73/10, +0.27)
```

```
$ pytest
======================== test session starts ========================
collected 19 items

tests/test_model.py ............                                 [ 63%]
tests/test_routes.py .......                                     [100%]

========================= 19 passed in 0.44s =========================
```

The raw output is committed under `docs/evidence/`, including the original 7.50/10 report and the failing test run described in section 11.

---

10. GitHub Actions workflow runs

`.github/workflows/ci.yml` runs on every push and pull request to `main`. It checks out the repository, sets up Python 3.11 with a pip cache, installs the application and development dependencies, runs the pre-commit hooks, runs Pylint and then runs pytest. A passing run takes about 47 seconds.

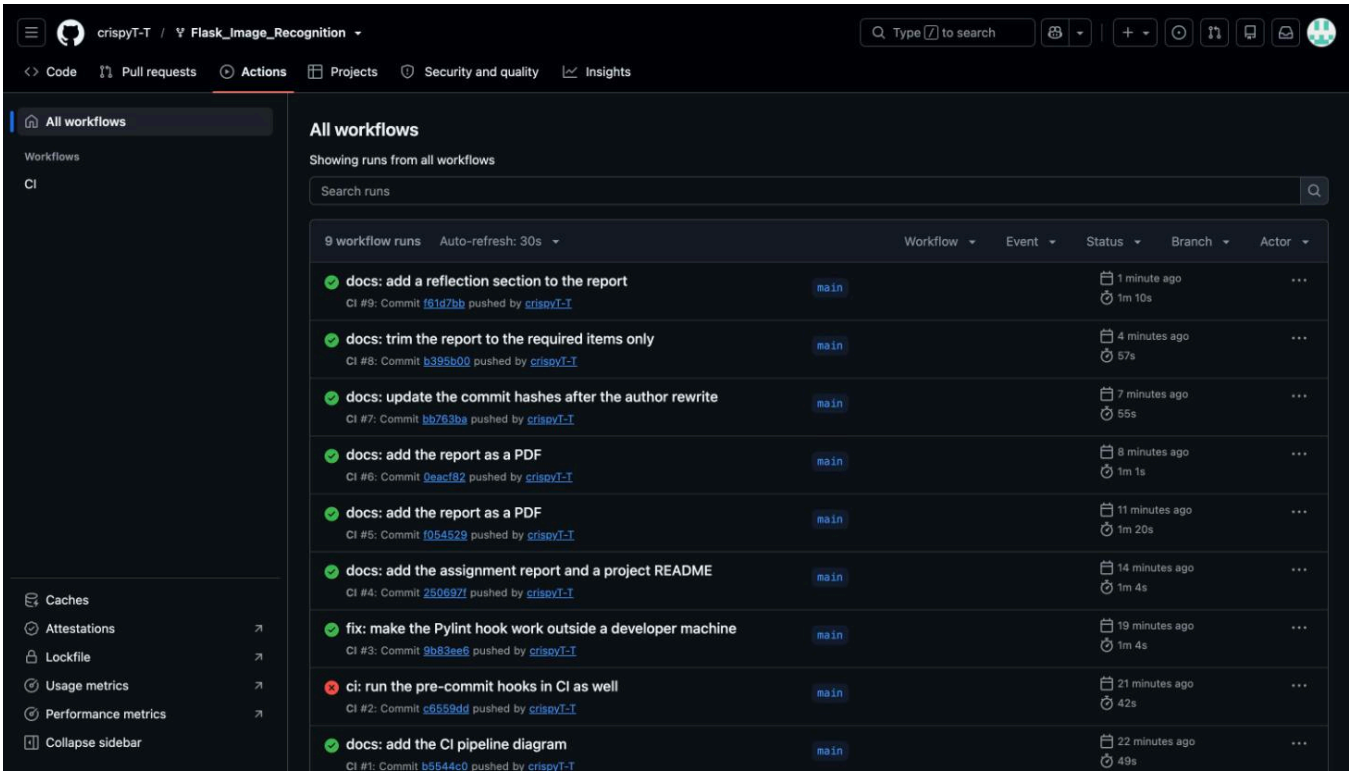


Figure 4. The workflow runs. #1 passed, #2 failed, and #3 passed after the fix described in section 11. Runs #4 onwards are the documentation commits.

---

11. A failed check and how it was corrected

Run **#2** failed. That commit added `pre-commit run --all-files` as a CI step, on the basis that CI should not assume the local hooks had actually run, since they can be skipped with `--no-verify`. The step failed immediately:

```
pylint...............................................................Failed
- hook id: pylint
Executable `.venv/bin/pylint` not found
```

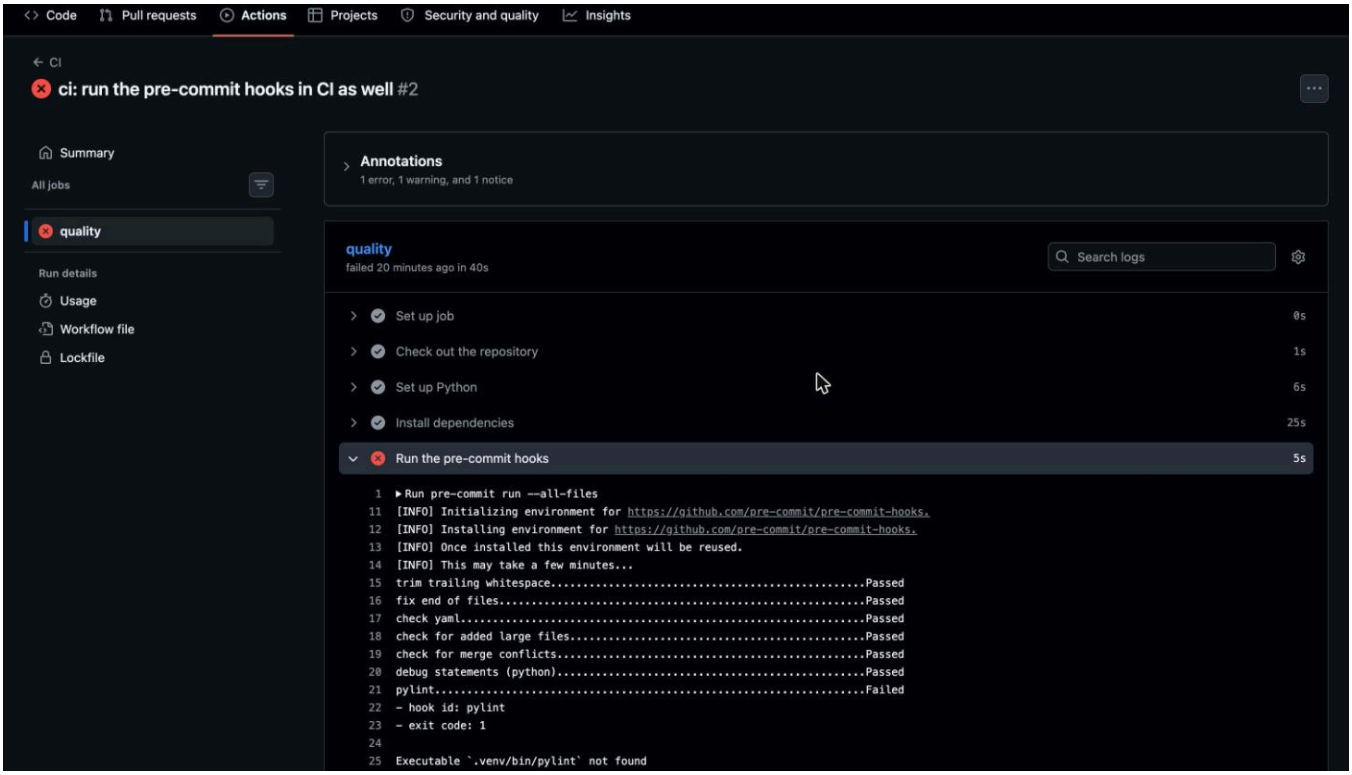


Figure 5. Run #2. The pre-commit step failed and everything after it was skipped.

The problem was a real mistake in my pre-commit configuration. I had written the Pylint hook as `entry: .venv/bin/pylint`. That path only exists on a machine where somebody has created a virtual environment in exactly that place. A CI runner installs the dependencies into whichever interpreter `setup-python` provides and has no `.venv` directory at all.

What makes this a good example is that the hook had worked on every commit up to that point. Nothing I could have run locally would have shown the problem, because the problem was that it only worked locally.

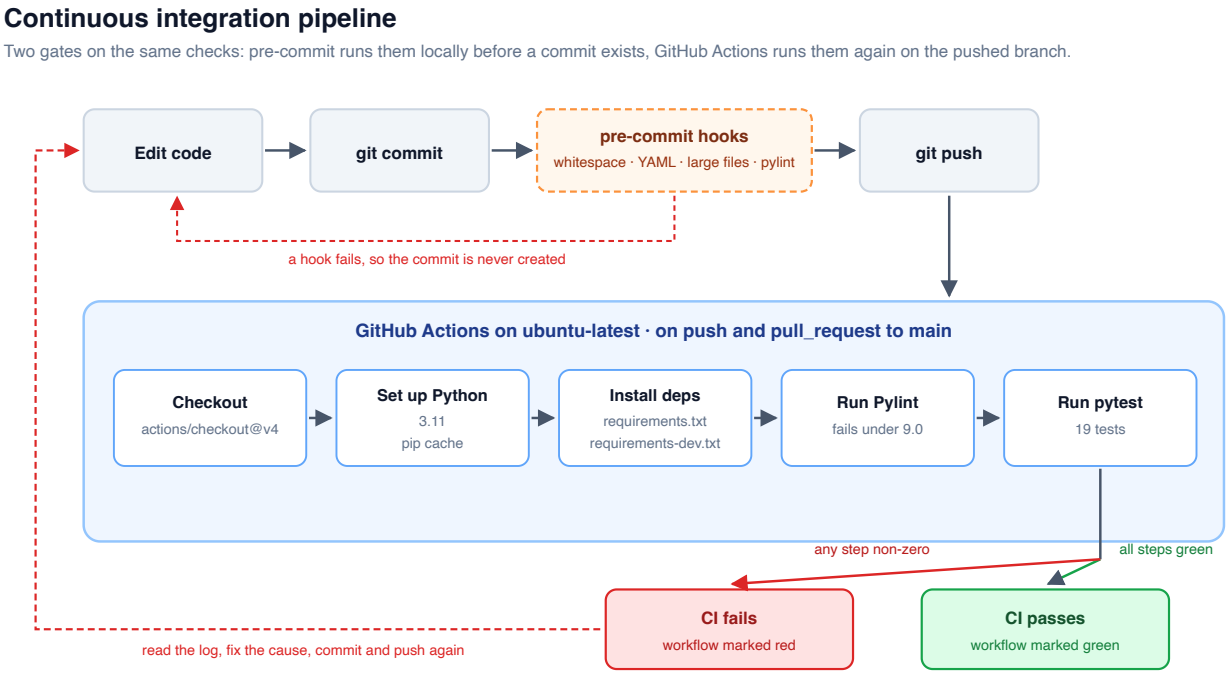
The fix was to call the module instead of a file path:

```
entry: python -m pylint
```

That resolves to the active virtual environment on my machine and to the runner's interpreter in CI, so both of them see Flask and Keras and resolve imports the same way. Run **#3** passed.

RUN	COMMIT	RESULT
#1	`807e663` docs: add the CI pipeline diagram	Passed
#2	`7d154b6` ci: run the pre-commit hooks in CI as well	**Failed**
#3	`259f2c0` fix: make the Pylint hook work outside a developer machine	Passed

12. CI pipeline diagram



The same checks run at two points. Pre-commit runs them on my machine before a commit is created, and GitHub Actions runs them again on the branch after it is pushed. The second gate is there because the first one can be skipped with `--no-verify`, so CI should not take it on trust.

---

13. Reflection

The legacy application could not run anywhere except its author's machine. Between the hardcoded `D:/SACHIN/...` path and dependencies with no builds for the current platform, neither problem could be seen by reading the code. Both only appeared when I tried to run it. A CI pipeline would have caught them on the first push, years ago.

The Pylint score was the least useful signal it gave me. The docstrings moved the number the most and mattered the least. The bare `except` barely moved it and mattered most. The score counts how many messages are left, not how serious they were, so chasing it would have meant doing the cheap work and stopping.

CI proved its worth immediately. The one genuine failure, the Pylint hook pointing at a virtual environment path, was a problem that no amount of local testing could have found, because the problem itself was the dependence on one particular machine. It showed up on the very first run with the hooks enabled, which makes the point better than I could have planned: a check that only ever runs in one place is not really a check.